

Brief Report – Pydantic Validation in the ChatGPT API Workflow

How Pydantic validation works

In my project, I used Pydantic to define clear data models that describe the expected structure of both input and output data.

Before calling the ChatGPT API, I validate the input using a Pydantic model.

If the input data does not match the required format (for example, missing fields or invalid values), the validation fails and the API call is skipped.

After receiving the response from the ChatGPT API, I validate the output JSON using another Pydantic model.

This ensures that the generated product listing follows a standardized schema with the correct fields and data types. If the output does not match the expected structure, it is rejected and handled as an error.

By validating both before and after processing, I ensure consistency and reliability throughout the entire workflow.

Benefits of validation

Using Pydantic validation provided several benefits in my project.

First, it prevented invalid or incomplete data from being sent to the API, reducing unnecessary API calls. Second, it guaranteed that the output data always followed a consistent and predictable structure.

Validation also made the code easier to debug, as Pydantic provides clear and informative error messages. Additionally, defining the data structure through models improved the readability and maintainability of the code.

Challenges you faced

One of the main challenges I faced was understanding how Python imports work, particularly the difference between modules and packages. Initially, the models.py file was not placed in the correct directory, which caused import errors.

Another challenge was handling cases where the API response was not valid JSON or did not fully match the expected schema. This required careful error handling and output validation to ensure the workflow did not break.

Improvements you made

During development, I made several improvements to the project.

I added input validation before making API calls to ensure that only valid requests were processed. I also introduced output validation using Pydantic to enforce a standardized response format.

Additionally, I improved error handling by separating successful results from errors and exporting them into dedicated JSON files. These changes made the workflow more robust, easier to maintain, and closer to a production-ready solution.